

Experiment 3

Create a Cryptocurrency using Python and perform mining in the Blockchain created

Aim

To create a peer-to-peer cryptocurrency (“HadCoin”) using Python and demonstrate mining, transaction propagation, and multi-node consensus across a 3-node Blockchain network.

Theory

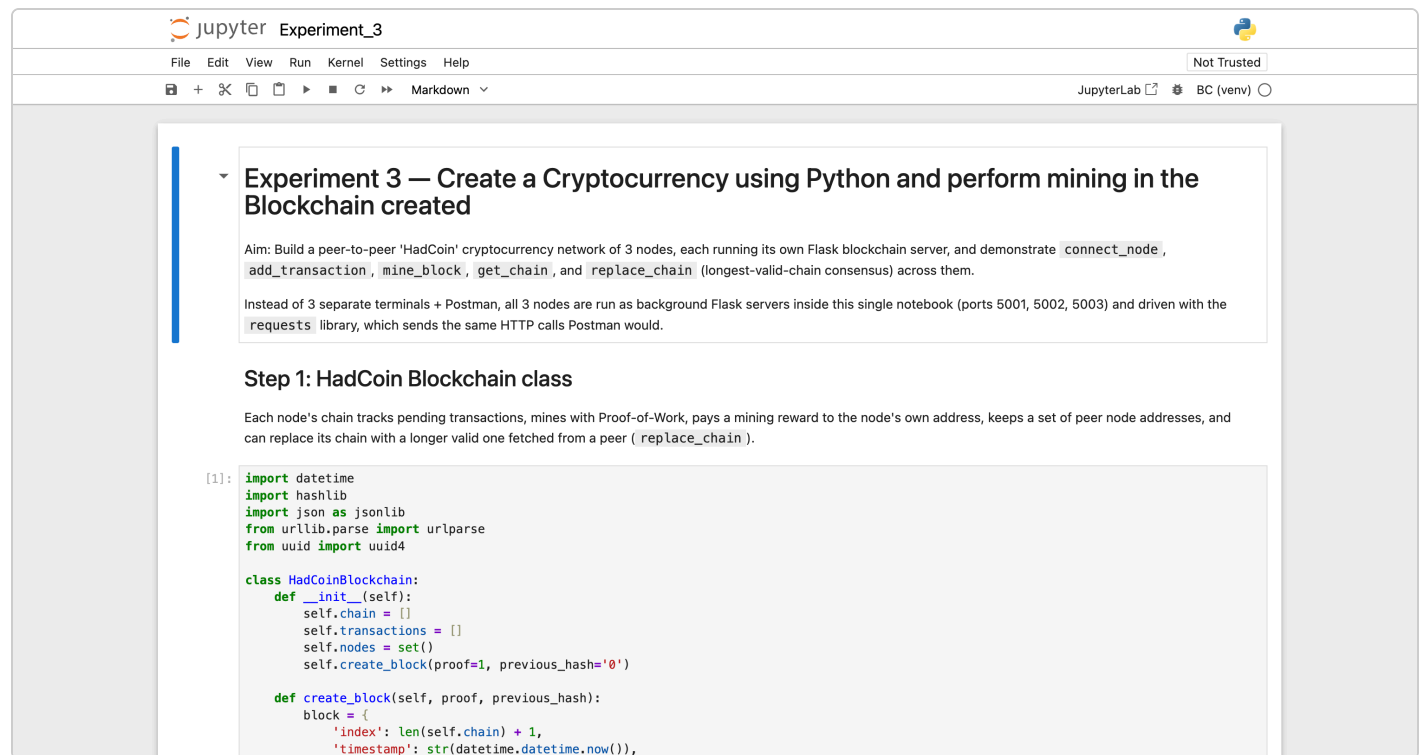
A real cryptocurrency network is not one blockchain process but many independent nodes, each holding its own copy of the chain and its own pool of pending transactions, communicating over HTTP. This experiment extends Experiment 2’s `Blockchain` class into a `HadCoinBlockchain` used by 3 separate Flask nodes (ports 5001, 5002, 5003), each identified by a UUID `node_address` used as its mining-reward address.

Each node exposes `connect_node` (register peer addresses), `add_transaction` (queue a transaction), `mine_block` (run Proof-of-Work, automatically append a mining-reward transaction to the node’s own address, and append the new block), `get_chain` (return the node’s current chain), and `replace_chain`. The last of these implements the **Longest Chain Rule**, the consensus mechanism that resolves forks in a P2P blockchain network without a central coordinator: a node asks every known peer for its chain, and if any peer’s chain is both longer and valid, the node adopts it in place of its own. This is essential because independent mining on different nodes inevitably produces temporarily divergent chains, and the network needs a deterministic rule for which version “wins”.

Rather than running `hadcoin_node_5001.py`, `hadcoin_node_5002.py`, `hadcoin_node_5003.py` in 3 separate terminals with a manual Postman session, all 3 Flask servers are run as background threads inside one notebook and driven with the `requests` library, which issues the exact same HTTP calls Postman would.

Tools used: Python 3.12, `hashlib`, `datetime`, `uuid`, `urllib.parse`, `Flask`, `requests`, run in a local Jupyter Notebook (`Experiment_3.ipynb`, in this folder).

Output



Experiment 3 — Create a Cryptocurrency using Python and perform mining in the Blockchain created

Aim: Build a peer-to-peer 'HadCoin' cryptocurrency network of 3 nodes, each running its own Flask blockchain server, and demonstrate `connect_node`, `add_transaction`, `mine_block`, `get_chain`, and `replace_chain` (longest-valid-chain consensus) across them.

Instead of 3 separate terminals + Postman, all 3 nodes are run as background Flask servers inside this single notebook (ports 5001, 5002, 5003) and driven with the `requests` library, which sends the same HTTP calls Postman would.

Step 1: HadCoin Blockchain class

Each node's chain tracks pending transactions, mines with Proof-of-Work, pays a mining reward to the node's own address, keeps a set of peer node addresses, and can replace its chain with a longer valid one fetched from a peer (`replace_chain`).

```
[1]: import datetime
import hashlib
import json as jsonlib
from urllib.parse import urlparse
from uuid import uuid4

class HadCoinBlockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.nodes = set()
        self.create_block(proof=1, previous_hash='0')

    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
```

Step 1 (start): the `HadCoinBlockchain` class — `create_block`, `get_previous_block`, and the start of `proof_of_work`, tracking `chain`, `transactions`, and `nodes` per instance.

```
jupyter Experiment_3

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab BC (venv)

self.chain.append(block)
return block

def get_previous_block(self):
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    new_proof = 1
    check_proof = False
    while not check_proof:
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof

def hash(self, block):
    encoded_block = jsonlib.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()
        ).hexdigest()
        if hash_operation[:4] != '0000':
            return False
        previous_block = block
        block_index += 1
```

Continuation of Step 1: `proof_of_work` (4-leading-zero target), `hash`, and `is_chain_valid`.

```
jupyter Experiment_3

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab BC (venv)

return self.get_previous_block()['index'] + 1

def add_node(self, address):
    parsed_url = urlparse(address)
    self.nodes.add(parsed_url.netloc)

def replace_chain(self, fetch_chain_fn):
    """fetch_chain_fn(node_address) -> (length, chain) or None if unreachable."""
    longest_chain = None
    max_length = len(self.chain)
    for node in self.nodes:
        result = fetch_chain_fn(node)
        if result is None:
            continue
        length, chain = result
        if length > max_length and self.is_chain_valid(chain):
            max_length = length
            longest_chain = chain
    if longest_chain:
        self.chain = longest_chain
    return True
return False

print("HadCoinBlockchain class defined.")

HadCoinBlockchain class defined.

Step 2: Spin up 3 nodes as Flask servers (ports 5001, 5002, 5003)

Each node gets its own node_address (a UUID, used as the miner's reward address) and its own HadCoinBlockchain instance, matching running hadcoin_node_5001.py, hadcoin_node_5002.py, hadcoin_node_5003.py in 3 separate terminals.

[2]: from flask import Flask, jsonify, request
import threading, time, requests

PORTS = [5001, 5002, 5003]
nodes_state = {} # port -> {'address': str, 'blockchain': HadCoinBlockchain, 'app': Flask}
```

Continuation of Step 1: `add_node` registers a peer's address, and `replace_chain` implements the Longest Chain Rule — adopting the longest valid chain found among peers.

```
jupyter Experiment_3
File Edit View Run Kernel Settings Help
Not Trusted
JupyterLab BC (venv)

blockchain = HadCoinBlockchain()
nodes_state[port] = {"address": node_address, "blockchain": blockchain}

@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    blockchain.add_transaction(sender=node_address, receiver='miner_reward', amount=1)
    block = blockchain.create_block(proof, previous_hash)
    return jsonify({'message': 'Congratulations, you just mined a block!', **block}), 200

@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({'chain': blockchain.chain, 'length': len(blockchain.chain)}), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    valid = blockchain.is_chain_valid(blockchain.chain)
    return jsonify({'message': 'valid' if valid else 'invalid'}), 200

@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    data = request.get_json()
    index = blockchain.add_transaction(data['sender'], data['receiver'], data['amount'])
    return jsonify({'message': f'Transaction will be added to Block {index}'}), 201

@app.route('/connect_node', methods=['POST'])
def connect_node():
    peers = request.get_json().get('nodes', [])
    for addr in peers:
        blockchain.add_node(addr)
    return jsonify({'message': 'Peers connected', 'total_nodes': list(blockchain.nodes)}), 201

@app.route('/replace_chain', methods=['GET'])
def replace_chain():
    def fetch(node_netloc):
        try:
```

Step 2: `mine_block`, `get_chain`, `is_valid`, `add_transaction`, and `connect_node` routes defined per-node, each backed by its own `HadCoinBlockchain` instance and `node_address`.

```
jupyter Experiment_3
File Edit View Run Kernel Settings Help
Not Trusted
JupyterLab BC (venv)

replaced = blockchain.replace_chain(fetch)
message = 'Chain was replaced with the longest peer chain.' if replaced else 'Chain is already the longest - nothing to update.'
return jsonify({'message': message, 'new_chain': blockchain.chain}), 200

return app

threads = []
for port in PORTS:
    app = make_app(port)
    t = threading.Thread(target=lambda a=app, p=port: a.run(host='127.0.0.1', port=p, use_reloader=False), daemon=True)
    t.start()
    threads.append(t)

time.sleep(1.5)
for port in PORTS:
    addr = nodes_state[port]['address']
    print(f"Node on port {port} -> address {addr}")

* Serving Flask app 'hadcoin_node_5001'
* Debug mode: off
* Serving Flask app 'hadcoin_node_5002'
* Debug mode: off
* Serving Flask app 'hadcoin_node_5003'
* Debug mode: off

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5003
Press CTRL+C to quit
Node on port 5001 -> address 518baa0e3335427fb503ed8a6b9ddb39
Node on port 5002 -> address a0ed50b733074c1c803d6be574c7e518
Node on port 5003 -> address bcd0cb7181d747df8cfe821621c05dbe

Step 3: Connect all 3 nodes to each other ( connect_node )

[3]: base_urls = {p: f"http://127.0.0.1:{p}" for p in PORTS}

for port in PORTS:
    peers = [base_urls[p] for p in PORTS if p != port]
```

The `replace_chain` route (using the Longest Chain Rule from Step 1) completes the Flask app, and all 3 nodes (5001, 5002, 5003) start successfully, each printing its own generated `node_address`.

```
jupyter Experiment_3
File Edit View Run Kernel Settings Help
Not Trusted
JupyterLab BC (venv)

127.0.0.1 -- [09/Sep/2026 11:50:58] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 -- [09/Sep/2026 11:50:58] "POST /connect_node HTTP/1.1" 201 -
connect_node on 5001: {'message': 'Peers connected', 'total_nodes': ['127.0.0.1:5003', '127.0.0.1:5002']}
connect_node on 5002: {'message': 'Peers connected', 'total_nodes': ['127.0.0.1:5001', '127.0.0.1:5003']}
connect_node on 5003: {'message': 'Peers connected', 'total_nodes': ['127.0.0.1:5001', '127.0.0.1:5002']}

Step 4: Add transactions and mine on node 5001

[4]: r = requests.post(f'{base_urls[5001]}/add_transaction', json={'sender': "a", "receiver": "b", "amount": 51})
print("add_transaction 1:", r.json())
r = requests.post(f'{base_urls[5001]}/add_transaction', json={'sender': "a", "receiver": "b", "amount": 52})
print("add_transaction 2:", r.json())
r = requests.post(f'{base_urls[5001]}/add_transaction', json={'sender': "a", "receiver": "b", "amount": 53})
print("add_transaction 3:", r.json())

r = requests.get(f'{base_urls[5001]}/mine_block')
print("\nmine_block on 5001:", r.json())

r = requests.get(f'{base_urls[5001]}/get_chain')
chain_5001 = r.json()
print(f"\n5001 chain length = {chain_5001['length']}")

127.0.0.1 -- [09/Sep/2026 11:50:58] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 -- [09/Sep/2026 11:50:58] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 -- [09/Sep/2026 11:50:58] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 -- [09/Sep/2026 11:50:58] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 -- [09/Sep/2026 11:50:58] "GET /get_chain HTTP/1.1" 200 -
add_transaction 1: {'message': 'Transaction will be added to Block 2'}
add_transaction 2: {'message': 'Transaction will be added to Block 2'}
add_transaction 3: {'message': 'Transaction will be added to Block 2'}

mine_block on 5001: {'index': 2, 'message': 'Congratulations, you just mined a block!', 'previous_hash': '85f488fe38fb6478564ccea92a79dc218d
f067b8ff3f5eafb3b6527a3d1a6fab', 'proof': 533, 'timestamp': '2026-09-09 11:50:58.539512', 'transactions': [{'amount': 51, 'receiver': 'b',
'sender': 'a'}, {'amount': 52, 'receiver': 'b', 'sender': 'a'}, {'amount': 53, 'receiver': 'b', 'sender': 'a'}, {'amount': 1, 'receiver': 'm
iner_reward', 'sender': '518baa0e3335427fb503ed8a6b9ddb39'}]}

5001 chain length = 2
```

Step 3: all 3 nodes register each other as peers via `connect_node`. Step 4: 3 transactions are queued and mined on node 5001 alone — the mined block includes a mining-reward transaction to 5001's own address, and its chain grows to length 2 while 5002/5003 remain at length 1.

```
jupyter Experiment_3 Last Checkpoint: 9 minutes ago
File Edit View Run Kernel Settings Help
Not Trusted
JupyterLab BC (venv)

127.0.0.1 -- [09/Sep/2026 11:50:58] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 -- [09/Sep/2026 11:50:58] "GET /replace_chain HTTP/1.1" 200 -
127.0.0.1 -- [09/Sep/2026 11:50:58] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 -- [09/Sep/2026 11:50:58] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 -- [09/Sep/2026 11:50:58] "GET /get_chain HTTP/1.1" 200 -
Node 5001 chain length BEFORE replace_chain: 2
Node 5002 chain length BEFORE replace_chain: 1
Node 5003 chain length BEFORE replace_chain: 1

replace_chain on 5002: Chain was replaced with the longest peer chain.
replace_chain on 5003: Chain was replaced with the longest peer chain.

Node 5001 chain length AFTER replace_chain: 2
Node 5002 chain length AFTER replace_chain: 2
Node 5003 chain length AFTER replace_chain: 2

Observations

• Each node keeps its own independent copy of the chain and its own pending-transaction pool, matching a real P2P blockchain network rather than a single shared process.
• Mining on only one node (5001) immediately made the 3 chains diverge in length (2 vs 1 vs 1), demonstrating the fork condition the Longest Chain Rule exists to resolve.
• Calling /replace_chain on the shorter nodes pulled and validated 5001's longer chain, bringing all 3 nodes back into consensus without any central coordinator.
• The mining reward transaction ( node_address -> miner_reward : 1 ) that mine_block adds automatically shows how miners are incentivized to add blocks, distinct from user-submitted transactions.
```

Step 5: chain lengths diverge (5001=2, 5002=1, 5003=1) before `replace_chain`; calling it on 5002 and 5003 pulls 5001's longer chain, bringing all 3 nodes to length 2 — followed by the written observations.

Conclusion

This experiment built a genuine 3-node peer-to-peer cryptocurrency network and demonstrated the exact failure mode — and its resolution — that P2P blockchains are designed around. Mining on only one node (5001) immediately produced divergent chain lengths across the network (2 vs. 1 vs. 1), because each node had only its own local view of the chain. Calling `/replace_chain` on the two shorter nodes resolved this without any central coordinator: each node independently fetched peer chains, validated them, and adopted 5001's longer chain, bringing all three back into consensus at length 2. The automatic mining-reward transaction (`node_address -> miner_reward : 1`) added inside `mine_block` further illustrates the economic incentive that drives miners to extend the chain, distinct from the user-submitted transactions being settled.